



UNIVERSITATEA DE VEST DIN TIMIȘOARA
FACULTATEA DE INFORMATICĂ
PROGRAMUL DE STUDII DE LICENȚĂ: Informatică

LUCRARE DE LICENȚĂ

COORDONATOR:
Lector Dr. Popovici Adriana

ABSOLVENT:
Preda Slavoliub-Denis

TIMIȘOARA
2026

UNIVERSITATEA DE VEST DIN TIMIȘOARA
FACULTATEA DE INFORMATICĂ
PROGRAMUL DE STUDII DE LICENȚĂ: Informatică

Dezvoltarea unei platforme web de management al proiectelor cu funcționalități asistate de inteligența artificială

COORDONATOR:
Lector Dr. Popovici Adriana

ABSOLVENT:
Preda Slavoliub-Denis

TIMIȘOARA
2026

Abstract

This paper presents a web platform for software project management. The platform covers various processes of the methodologies that can be used with Scrum, Kanban, or Scrumban. Thus, a user can benefit from a customizable release assistant, a recommendation system, or a methodology including mini-explanations using Artificial Intelligence.

The application covers tasks and subtasks, a *drag & drop* Kanban board, sprint management (when necessary), user roles and permissions, and an audit log for critical actions. The most important functionalities include the ability to switch project methodologies (Scrum, Kanban, Scrumban) without losing tasks, and the on-the-fly adjustment of the user interface (for example, Backlog/Sprint pages appear/disappear depending on the methodology). The application can be integrated with GitHub via webhooks, so that commits and pull requests can be associated with a task. Artificial Intelligence components are used in concrete use cases, such as requirements refinement, estimation suggestions, workload balancing, or summary generation.

Cuprins

1	Introducere	5
1.1	Context și motivație	5
1.2	Problema abordată	5
1.3	Soluția propusă	6
1.4	Structura lucrării	6
2	Aplicații existente	7
2.1	Jira Software	7
2.2	Trello	7
2.3	GitHub Projects	7
2.4	Concluzie	8
3	Descrierea aplicației	9
3.1	Cerințe funcționale	9
3.2	Cazuri de utilizare	9
3.3	Diagramă arhitectură	9
3.4	Interfața grafică	14
3.5	Diagramă navigare	15
3.6	Arhitectura aplicației	15
3.7	Modelul de date	15
3.8	Tehnologii folosite	15
4	Contribuția proprie	16
4.1	Funcționalități implementate	16
4.2	Detalii de implementare	19
4.3	Validarea funcționalităților implementate	21
	Bibliografie	23

Capitolul 1

Introducere

1.1 Context și motivație

Într-un proiect software, nu toți membrii echipei lucrează identic. Unii sunt potriviți pentru a planifica în sprinturi și cu obiective care sunt precise. Alții, la cealaltă extremă, au urgențe și schimbări aproape toată ziua și beneficiază de un flux de lucru stabil. În practică, acest lucru se face uneori din obișnuință, iar ei nu pot avea un sistem rigid.

În plus, următoarele probleme recurente: sarcini mutate sau șterse fără a ști cine a făcut ce; estimări modificate, fără un motiv clar; documentație rămasă în urmă; pierdere de timp cu scrierea descrierilor, a criteriilor de acceptare sau a rapoartelor. Scopul meu pentru această lucrare de licență este de a construi o soluție care să ajute la organizare (sarcini, panou, planificarea sprinturilor) și control (utilizatori, roluri, permisiuni, istoric) construind funcționalități bazate pe inteligența artificială acolo unde ajută cu adevărat la economisirea timpului și la reducerea incertitudinilor, o direcție susținută de cercetările recente privind viitorul dezvoltării software [1].

1.2 Problema abordată

Problema poate fi formulată simplu: cum se poate construi o platformă web de management al proiectelor software care:

- se adaptează la metodologiile Scrum, Kanban și Scrumban;
- permite schimbarea metodologiei în același proiect fără pierderea datelor și fără să se strice fluxul;
- păstrează un istoric clar al modificărilor și controlează acțiunile sensibile ale utilizatorilor;
- integrează inteligența artificială și GitHub în fluxul de lucru, nu ca funcții separate.

1.3 Soluția propusă

Soluția constă în aplicația *SDLC Hub*. Utilizatorul se înregistrează, își verifică e-mailul, se conectează, se alătură unui proiect (prin invitație sau cod) sau creează unul nou. În timpul creării proiectului, este posibilă utilizarea unui asistent pentru a seta metodologia recomandată, iar SDLC Hub adaptează structura aplicației și modulele care sunt active și utilizabile (Backlog, Sprints). Funcționalitățile sunt: panou, sarcini, subsarcini, roluri, permisiuni, jurnal de audit, integrare GitHub și unele capacități de inteligență artificială (procesarea cerințelor, estimări și generarea de rapoarte).

1.4 Structura lucrării

Capitolul 2 prezintă trei aplicații existente și o analiză scurtă. Capitolul 3 descrie aplicația realizată: cerințe, cazuri de utilizare, funcționalități implementate, interfață, arhitectură și model de date.

Capitolul 2

Aplicații existente

2.1 Jira Software

Jira este una dintre cele mai folosite soluții pentru echipe agile, mai ales în firme medii și mari. Este bună când ai nevoie de reguli, rapoarte și configurări detaliate.

Avantaje:

- suport complet pentru Scrum (backlog, sprint-uri, rapoarte);
- workflow-uri configurabile și multe setări pentru proiecte mari;
- roluri și permisiuni configurabile.

Dezavantaje:

- pentru proiecte mici poate fi prea complexă și consumă timp la setare;
- dacă se personalizează prea mult, devine greu de întreținut;
- utilizatorii ajung să se „încurce” în opțiuni și ecrane.

2.2 Trello

Trello este cunoscut pentru simplitate și pentru ideea de board cu liste și carduri. E bun pentru echipe mici și proiecte mai simple.

Avantaje:

- ușor de învățat și de folosit;
- potrivit pentru un Kanban simplu;
- vizualizare rapidă a progresului.

Dezavantaje:

- limitat la partea de control (permisiuni granulare) și trasabilitate (audit);
- funcțiile mai avansate depind de extensii;
- nu e orientat pe sprint management complet.

2.3 GitHub Projects

GitHub Projects este atractiv pentru echipe tehnice deoarece este aproape de cod: issues, pull request-uri și automatizări.

Avantaje:

- se leagă direct de repository și de dezvoltarea efectivă;
- automatizări pe evenimente (issue/PR);
- trasabilitate bună între task și activitatea din cod.

Dezavantaje:

- mai greu de folosit pentru persoane non-tehnice;
- partea de rapoarte agile e mai simplă decât în tool-uri dedicate;
- depinde mult de disciplina echipei (issues/PR scrise și legate corect).

2.4 Concluzie

Observăm din exemplele de mai sus că unele instrumente sunt prea greoaie pentru echipele mici, iar altele sunt prea simple pentru auditare și control. Trecerea fluidă și adaptabilitatea între metodologii precum Scrum și Kanban reprezintă o necesitate demonstrată pentru optimizarea performanței echipelor [2]. De asemenea, nu este întotdeauna clar dacă ne schimbăm abordarea în cadrul aceluiași proiect, iar inteligența artificială existentă este separată de pașii muncii reale. SDLC Hub încearcă să îmbine gestionarea muncii cu cadrul de „control”, integrarea cu GitHub și inteligența artificială încorporată în proces.

Capitolul 3

Descrierea aplicației

3.1 Cerințe funcționale

Aplicația trebuie să permită:

- creare cont, confirmare e-mail, autentificare și recuperare parolă;
- acces într-un proiect prin invitație sau cod și creare proiect nou;
- recomandare metodologie prin wizard și configurare automată;
- schimbarea metodologiei proiectului fără pierdere de date;
- gestionare sarcini și sub-sarcini, mutare pe board și listă;
- management sprint (unde este activ), rapoarte și notificări;
- roluri, permisiuni și audit log;
- integrare GitHub prin webhooks;
- funcții AI pentru cerințe, estimări, încărcare și rezumate.

3.2 Cazuri de utilizare

În continuare sunt descrise principalele acțiuni pe care le face un utilizator în aplicație.

3.3 Diagramă arhitectură

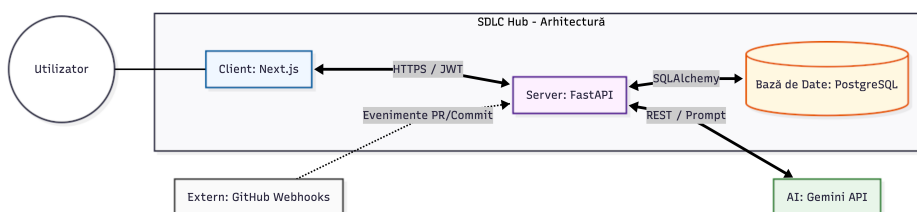


Figura 3.1: Diagrama Arhitecturii Sistemului SDLC Hub

3.3.1 Înregistrare utilizator

Actori: Utilizator

Precondiții: utilizatorul este pe pagina de înregistrare.

Pași:

1. Completează nume, prenume, e-mail, parolă, confirmare parolă.
2. Sistemul validează datele.
3. Sistemul creează contul în stare neconfirmată.
4. Sistemul trimite e-mail de confirmare.

Alternative: e-mail folosit deja / date invalide → mesaj de eroare.

Postcondiții: cont creat, dar neactiv până la confirmare.

3.3.2 Confirmare e-mail

Actori: Utilizator

Precondiții: cont neconfirmat.

Pași:

1. Accesează link-ul sau introduce codul primit.
2. Sistemul verifică token-ul/codul.
3. Sistemul activează contul.

Alternative: token invalid/expirat → retrimiteră confirmare.

Postcondiții: cont activ.

3.3.3 Autentificare

Actori: Utilizator

Precondiții: cont activ.

Pași:

1. Introduce e-mail și parolă.
2. Sistemul verifică datele.
3. Sistemul generează token JWT și îl trimite către interfață.

Alternative: parolă greșită / cont neconfirmat → eroare.

Postcondiții: utilizator autentificat.

3.3.4 Recuperare și resetare parolă

Actori: Utilizator

Precondiții: utilizatorul are cont.

Pași:

1. Cere resetare parolă (introduce e-mailul).
2. Sistemul trimite link/token de resetare.
3. Utilizatorul setează o parolă nouă.

Postcondiții: parola este schimbată și utilizatorul se poate autentifica.

3.3.5 Prima intrare în aplicație (invitație sau cod)

Actori: Utilizator

Precondiții: utilizator autentificat.

Pași:

1. Sistemul verifică dacă există invitații pentru e-mail.
2. Dacă există, utilizatorul acceptă și intră în proiect.
3. Dacă nu există, utilizatorul poate introduce un cod de proiect primit de la un administrator de proiect.

Alternative: cod invalid/expirat → eroare.

Postcondiții: utilizator membru într-un proiect.

3.3.6 Creare proiect

Actori: Administrator proiect (*Project Admin*)

Precondiții: utilizator autentificat.

Pași:

1. Completează nume proiect, cheie proiect, descriere și logo (opțional).
2. Sistemul verifică unicitatea cheii proiectului.
3. Sistemul creează proiectul și utilizatorul devine administrator.

Postcondiții: proiect creat.

3.3.7 Recomandare metodologie (wizard)

Actori: Administrator proiect

Precondiții: proiect existent sau în creare.

Pași:

1. Pornește wizard-ul de recomandare.
2. Răspunde la întrebări (echipă, tip muncă, schimbări, experiență, obiectiv).
3. Sistemul calculează scorul intern pentru Scrum/Kanban/Scrumban.
4. Sistemul cere o explicație scurtă de la AI.
5. Utilizatorul acceptă recomandarea sau alege manual.
6. Sistemul configurează proiectul (module, pagini, workflow).

Postcondiții: proiect configurat, interfața se adaptează.

3.3.8 Schimbarea metodologiei proiectului

Actori: Administrator proiect

Precondiții: proiect existent, drepturi de administrare.

Pași:

1. Selectează o metodologie nouă din setările proiectului.
2. Sistemul arată ce se schimbă (Backlog/Sprints, reguli, meniuri).
3. Utilizatorul confirmă.
4. Sistemul păstrează sarcinile și face maparea status-urilor pe un workflow compatibil.
5. Dacă se trece pe Kanban, sprint-urile sunt închise/arhivate, iar sarcinile rămân pe board.

6. Dacă se trece pe Scrum, se activează Backlog și Sprints, iar sarcinile neplanificate pot fi tratate ca backlog.
7. Se salvează o intrare în audit log pentru schimbarea metodologiei.

Postcondiții: proiectul funcționează pe noua metodologie, datele rămân.

3.3.9 Invitare membri și setare rol

Actori: Administrator proiect

Precondiții: proiect existent.

Pași:

1. Introduce e-mailurile membrilor.
2. Alege rolul fiecăruia (de exemplu: Tech Lead, Developer, PO/PM, Scrum Master/Flow Manager).
3. Sistemul trimite invitații.
4. La acceptare, membrii sunt adăugați cu rol.

Postcondiții: membrii apar în proiect și au permisiunile corespunzătoare.

3.3.10 Creare și editare sarcină (cu sub-sarcini)

Actori: Administrator proiect / PO/PM / Scrum Master/Flow Manager (în funcție de permisiuni)

Precondiții: utilizator membru în proiect.

Pași:

1. Completează titlu, descriere (rich text), prioritate.
2. Adaugă story points (dacă se folosesc).
3. Adaugă sub-sarcini (checklist).
4. Salvează sarcina.

Postcondiții: sarcina apare pe board și în listă.

3.3.11 Mutare sarcină pe board și audit log

Actori: Developer / QA / Manager (în funcție de permisiuni)

Precondiții: sarcină existentă.

Pași:

1. Mută sarcina între coloane (drag & drop).
2. Sistemul verifică tranziția.
3. Sistemul salvează status-ul nou.
4. Sistemul adaugă intrare în audit log (cine, ce, când).

Alternative: tranziție interzisă → mutarea este blocată.

Postcondiții: status actualizat și logat.

3.3.12 Guvernanța sarcinilor (ștergere / re-estimare controlată)

Actori: Administrator proiect / Tech Lead / PO/PM / Scrum Master

Precondiții: sarcină existentă.

Pași:

1. Utilizatorul încearcă ștergere sau schimbare estimare.
2. Sistemul verifică rolul și permisiunile.
3. Sistemul cere confirmare.
4. Sistemul aplică acțiunea și o salvează în audit log.

Postcondiții: acțiune aplicată controlat și trasabil.

3.3.13 Management sprint (Scrum/Scrumban)

Actori: Scrum Master / Administrator proiect

Precondiții: metodologie Scrum sau Scrumban activă.

Pași:

1. Creează sprint (perioadă și obiectiv).
2. Planifică sarcini în sprint.
3. Porneste sprint-ul.
4. Închide sprint-ul.

Postcondiții: sprint finalizat, date păstrate pentru rapoarte.

3.3.14 Funcții AI

Actori: Utilizator (cu acces AI)

Precondiții: proiect activ, cheie AI configurată.

Pași (general):

1. Utilizatorul apasă butonul de AI în ecranul relevant.
2. Sistemul trimite date către AI.
3. Rezultatul apare în interfață și poate fi editat înainte de salvare.

Funcții:

- **Spec Refiner:** generează user story, criterii de acceptare și propune sub-sarcini. Utilizarea modelelor mari de limbaj (LLM) în ingineria cerințelor s-a dovedit extrem de eficientă pentru structurarea specificațiilor inițiale [3].
- **Poker Estimator:** sugerează story points pe baza descrierii, o abordare validată de studiile recente privind estimarea automată a efortului prin tehnici de inteligență artificială [4].
- **Workload Balancer:** analizează încărcarea și sugerează redistribuire.
- **Release Notes:** generează rezumat pentru sprint.

Postcondiții: conținut generat și verificat de utilizator.

3.3.15 Integrare GitHub (webhooks)

Actori: GitHub, Sistem

Precondiții: webhook configurat și proiect conectat.

Pași:

1. GitHub trimite eveniment (commit sau pull request).

2. Sistemul caută cheia sarcinii în mesaj (ex. KEY-123).
3. Sistemul atașează evenimentul la sarcină (comentariu/istoric).
4. La PR Open, sarcina poate fi mutată automat în „Code Review”.
5. Se salvează audit log pentru acțiuni automate.

Postcondiții: legătură între cod și sarcini, trasabilitate mai bună.

3.3.16 Rapoarte și notificări

Actori: Utilizator

Precondiții: proiect existent.

Descriere:

- grafic de viteză (*Velocity Chart*) unde este relevant;
- export PDF pentru raport de stare;
- notificări în aplicație: mențiuni, asignări, alerte.

3.4 Interfața grafică

Interfața este făcută să fie cât mai clară: utilizatorul vede proiectul, board-ul și sarcinile. În funcție de metodologie, interfața se schimbă (de exemplu în Kanban nu apar sprint-urile).

Paginile principale:

- Landing Page
- Register
- Verify Email
- Login
- Forgot Password
- Reset Password
- First Login
- Project Wizard
- Dashboard
- Board
- Backlog
- Sprints
- Task Details
- Reports
- Documentation
- Role Permissions
- Project Settings
- Profile / Notifications

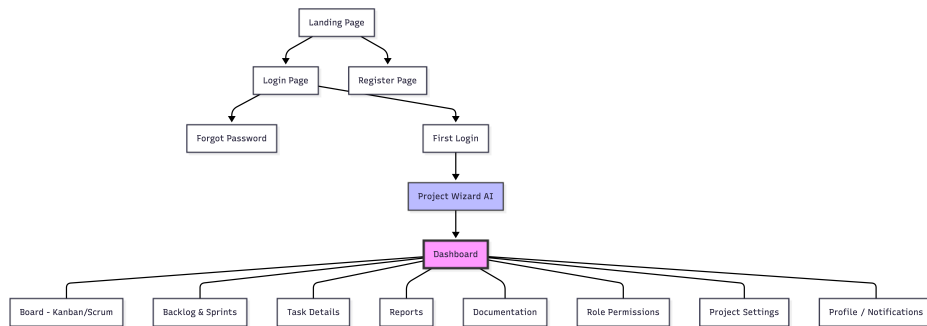


Figura 3.2: Diagrama Navigare

3.5 Diagramă navigare

3.6 Arhitectura aplicației

Aplicația este organizată pe trei niveluri: interfață (frontend), logică (backend) și date (bază de date). Frontend-ul comunică cu backend-ul prin API de tip REST. Backend-ul se ocupă de autentificare, reguli, permisiuni, audit log, AI și integrarea GitHub.

3.7 Modelul de date

Entitățile principale sunt: User, Project, Membership, Role, Task, Subtask, Sprint și AuditLog. Proiectul păstrează metodologia activă, iar sarcinile aparțin proiectului și pot aparține opțional unui sprint.

3.8 Tehnologii folosite

- Backend: Python, FastAPI, Uvicorn, SQLAlchemy, Alembic, PostgreSQL, JWT (python-jose), passlib[bcrypt], python-dotenv, fpdf2, integrare AI local.
- Frontend: Next.js, TypeScript, Tailwind CSS, shadcn/ui, TanStack Query, React Hook Form + Zod, dnd-kit, Recharts, Tiptap, Lucide React.
- DevOps: GitHub Webhooks, Git, GitHub, ngrok (local), Swagger UI / Postman.

Capitolul 4

Contribuția proprie

Acest capitol prezintă contribuția proprie realizată în cadrul aplicației *SDLC Hub*. Accentul este pus pe modulele proiectate și implementate, pe modul în care acestea colaborează între ele și pe validarea funcționalităților obținute. Lucrarea este una preponderent practică, iar contribuția principală constă în realizarea unei platforme web full-stack pentru managementul proiectelor software, cu suport pentru Scrum, Kanban și Scrumban, funcții asistate de inteligență artificială, auditare, permisiuni, raportare, integrare GitHub și colaborare în timp real.

4.1 Funcționalități implementate

În această secțiune sunt prezentate modulele implementate în aplicație. Pentru a păstra cuprinsul clar, modulele individuale sunt descrise ca subsecțiuni, deoarece ele reprezintă detalii ale contribuției practice, nu capitole independente.

Managementul metodologiilor Scrum, Kanban și Scrumban

Una dintre contribuțiile principale este tratarea metodologiei proiectului ca o proprietate activă a sistemului, nu doar ca o etichetă. În funcție de metodologia aleasă, aplicația modifică atât comportamentul backend-ului, cât și interfața grafică.

Pentru proiectele de tip Scrum și Scrumban sunt activate backlog-ul, sprinturile și câmpurile de estimare prin story points. Pentru proiectele Kanban, accentul cade pe fluxul continuu de lucru, iar elementele specifice sprinturilor sunt ascunse sau blocate. Astfel, aceeași aplicație poate fi folosită de echipe cu moduri diferite de organizare.

Am implementat afișarea dinamică a paginii Backlog doar pentru Scrum și Scrumban, blocarea operațiilor de sprint pentru Kanban, afișarea sau ascunderea câmpului *story points* în funcție de metodologie și adaptarea dashboard-ului la modul de lucru al proiectului.

Methodology Transition Wizard

O contribuție importantă este implementarea unui *Methodology Transition Wizard*, prin care metodologia unui proiect poate fi schimbată fără pierderea datelor. Funcționalitatea rezolvă o problemă practică: o echipă poate porni cu Scrum și poate ajunge ulterior la concluzia că un flux Kanban este mai potrivit, sau invers.

Pentru această funcționalitate am implementat un endpoint de previzualizare a impactului schimbării metodologiei și un endpoint de aplicare efectivă a schimbării. Previzualizarea întoarce blocaje, avertismente, acțiuni recomandate și statistici despre elementele afectate. De exemplu, trecerea de la Scrum la Kanban este blocată dacă există un sprint activ, pentru a evita migrarea nesigură a task-urilor.

În frontend, schimbarea metodologiei nu se face printr-un simplu selector salvat direct. Utilizatorul vede un modal dedicat, cu impactul tranziției, blocaje, avertismente și acțiuni automate. După confirmare, backend-ul aplică tranziția și salvează evenimentul în jurnalul de audit al proiectului.

Sistemul de roluri și permisiuni

Am implementat un sistem de permisiuni care controlează acțiunile utilizatorilor în funcție de rolul lor în proiect. Permisunile sunt definite prin chei clare, precum:

- TASK_CREATE, TASK_UPDATE, TASK_MOVE;
- SPRINT_CREATE, MEMBER_INVITE, ROLE_MANAGE;
- SETTINGS_MANAGE, AI_USE, TEAM_MANAGE.

Verificarea permisiunilor este realizată în backend, pentru ca aplicația să nu depindă doar de ascunderea butoanelor în interfață. În același timp, frontend-ul este *permission-aware*: butoanele și acțiunile indisponibile utilizatorului curent sunt ascunse sau dezactivate. Astfel, un utilizator fără drepturi nu poate crea sprinturi, modifica roluri, invita membri sau schimba setările proiectului.

Audit log și trasabilitate

Pentru acțiunile importante am implementat auditare. Sistemul salvează evenimente relevante produse în proiect, cum ar fi modificări de task, schimbări de status, comentarii, schimbări de estimare, generare de documentație, modificarea metodologiei și acțiuni automate venite din GitHub.

Auditarea permite urmărirea întrebărilor practice: cine a schimbat statusul unui task, cine a modificat estimarea, când a fost generată documentația, ce eveniment GitHub a mutat un task în *Review* sau *Done* și cine a aplicat schimbarea metodologiei proiectului.

Managementul task-urilor, subtasks și comentariilor

Am implementat modulul central al aplicației: gestionarea sarcinilor. Un task poate avea titlu, descriere, status, prioritate, persoană asignată, story points, dată limită, echipă asociată, subtasks și comentarii. Task-urile pot fi create din dialogul de creare, editate în pagina de detalii și mutate pe board prin *drag and drop*.

Descrierea task-urilor folosește un editor Rich Text bazat pe Tiptap, ceea ce permite structurarea cerințelor prin liste, text evidențiat, blocuri de cod și link-uri. Comentariile pot fi editate sau șterse de autor, iar modificările importante sunt înregistrate în audit log.

Backlog, sprinturi și board

Pentru Scrum și Scrumban am implementat backlog-ul, crearea sprinturilor, pornirea sprintului și închiderea sprintului. Board-ul este componenta vizuală principală pentru

urmărirea execuției. Pentru Scrum și Scrumban, board-ul poate fi legat de sprintul activ, iar pentru Kanban funcționează ca flux continuu.

Această implementare permite folosirea aplicației atât în echipe care lucrează iterativ, cât și în echipe care preferă un flux continuu.

Funcționalități asistate de inteligență artificială

Am integrat funcții AI direct în fluxurile de lucru ale aplicației. AI-ul este folosit pentru recomandarea metodologiei, sugestii de roluri, generarea descrierii task-urilor, rafinarea specificațiilor, generarea criteriilor de acceptare, propunerea de subtasks, estimarea story points, analiza riscurilor, rafinarea sugestiilor de redistribuire și generarea de release notes.

Funcționalitățile AI sunt implementate astfel încât rezultatele să fie vizibile și editabile de utilizator înainte de salvare. În lipsa unei chei API externe, serviciul AI poate folosi fallback local bazat pe reguli simple, pentru ca aplicația să rămână utilizabilă în mediul local de testare.

Workload Balancer

Am implementat o pagină dedicată pentru analiza încărcării membrilor echipei. Modulul calculează pentru fiecare membru numărul de task-uri active, story points, task-uri întârziate, task-uri în review și un scor de risc.

Pe baza acestor date, aplicația poate sugera redistribuirea task-urilor. Sugestiile sunt calculate pe baza regulilor și pot fi explicate cu ajutorul AI-ului. În interfață, utilizatorul poate vedea membrii supraîncărcați și poate redistribui task-uri între membri.

Raportare și export PDF

Pentru monitorizarea progresului am implementat un modul de raportare în pagina `/dashboard/reports`. Acesta include metrice de overview, distribuția task-urilor după status și prioritate, velocity chart, sprint burndown, cumulative flow, lead time pe priorități și export PDF pentru raportul de stare al proiectului.

Exportul PDF este util deoarece permite obținerea unui raport extern aplicației, care poate fi transmis sau arhivat.

Notificări, remindere și colaborare în timp real

Am implementat un sistem de notificări în aplicație, cu unread count în sidebar și pagină dedicată. Notificările sunt generate pentru asignări, mențiuni, invitații în calendar, task-uri aproape de termen, task-uri întârziate și riscuri detectate.

Pentru colaborarea în timp real am implementat WebSocket. Backend-ul transmite evenimente precum `notification.created`, `task.changed`, `calendar.changed`, `sprint.changed`, `user.updated` și `presence.changed`. Frontend-ul ascultă aceste evenimente și actualizează paginile relevante fără refresh complet.

Calendar și evenimente recurente

Am implementat un calendar de proiect care afișează atât evenimentele echipei, cât și task-urile pe baza datei limită. Calendarul are vizualizări de tip month, week, day

și agenda. Utilizatorii pot crea evenimente cu titlu, tip, interval orar, link, locație, descriere și participanți.

Am implementat și evenimente recurente, cu moduri *daily*, *weekdays* și *weekly*, precum și preset pentru *Daily Standup*. Pentru proiectele Scrum și Scrumban, începutul și finalul sprinturilor apar ca milestone-uri virtuale în calendar.

Documentație internă și automatizare

Am implementat modulul de documentație internă disponibil în pagina `/dashboard/documentation`. Acesta permite crearea, editarea și ștergerea paginilor de documentație.

O parte importantă este generarea automată a documentației din task-uri finalizate. Când un task ajunge în statusul *Done*, aplicația poate genera sau actualiza o pagină de documentație asociată. Același mecanism este folosit și când un pull request GitHub este închis cu merge și mută task-ul în *Done*.

Integrarea GitHub și fluxul DevOps

Am implementat integrarea cu GitHub prin webhooks. Backend-ul primește evenimente GitHub, verifică semnătura webhook-ului și caută cheia task-ului în mesajele de commit sau în evenimentele de pull request.

Commit-urile pot fi atașate automat ca evenimente sau comentarii la task-uri. Un pull request deschis poate muta task-ul în *Review*, iar un pull request închis cu merge poate muta task-ul în *Done*. Astfel, aplicația leagă cerințele și task-urile de activitatea reală din cod.

Ierarhie de echipe

Am implementat un modul pentru organizarea membrilor în echipe și subechipe. Aplicația permite crearea echipelor, asocierea membrilor la echipe, asocierea task-urilor cu o echipă de livrare și afișarea arborelui ierarhic al echipelor.

Pentru rolurile de coordonare există și o vedere *My team view*, care evidențiază echipa proprie, membrii direcți, subechipele și task-urile asociate.

Profil utilizator, avatar și branding

Am implementat pagina de account settings, unde utilizatorul își poate modifica numele, e-mailul, parola și avatarul. Avatarul este încărcat local și este afișat în sidebar, board, backlog, task detail, comentarii, audit log, calendar, team și activity center.

Am adăugat și elemente de branding: componentă vizuală pentru brand, iconiță de aplicație, favicon, manifest web și actualizarea paginii publice și a sidebar-ului.

4.2 Detalii de implementare

Această secțiune descrie pe scurt modul tehnic în care au fost implementate modulele principale.

Implementarea backend

Backend-ul este construit în Python folosind FastAPI. Acesta expune endpoint-uri REST pentru autentificare, proiecte, task-uri, sprinturi, calendar, notificări, documentație, echipe, rapoarte și integrare GitHub. Pentru persistență am folosit SQLAlchemy, Alembic și PostgreSQL. Migrațiile Alembic permit evoluția bazei de date pe măsură ce apar funcționalități noi.

Regulile aplicației sunt verificate server-side. De exemplu, dacă un utilizator încearcă să modifice un task fără permisiune, backend-ul respinge cererea, chiar dacă utilizatorul ar încerca să o trimită manual.

Implementarea frontend

Frontend-ul este construit cu Next.js, TypeScript și Tailwind CSS. Aplicația este organizată pe rute precum dashboard, board, backlog, task detail, team, settings, calendar, reports, workload, notifications, documentation și devops.

Interfața adaptează meniurile și acțiunile în funcție de metodologia proiectului și de permisiunile utilizatorului curent. De exemplu, pagina Backlog nu este afișată pentru Kanban, iar butoanele de creare sprint sau invitare membri apar doar pentru utilizatorii care au permisiunile necesare.

Implementarea verificării permisiunilor

Fiecare rol conține o listă de permisiuni, iar backend-ul verifică dacă utilizatorul curent are cheia necesară pentru operația solicitată. Fluxul general este: identificarea utilizatorului autentificat, identificarea proiectului, identificarea rolului în proiect, verificarea cheii de permisiune și executarea sau respingerea operației.

Implementarea tranziției între metodologii

Pentru tranziția între metodologii am separat analiza impactului de aplicarea efectivă. Endpoint-ul de preview calculează metodologia curentă, metodologia țintă, sprinturile active, sprinturile viitoare, task-urile afectate, blocajele și avertismentele. Endpoint-ul de apply aplică schimbarea într-o tranzacție și scrie audit log.

Implementarea funcțiilor AI

Funcțiile AI sunt centralizate într-un serviciu backend. Serviciul construiește prompturi, apelează providerul AI dacă este configurat și întoarce rezultate structurate către endpoint-uri. În lipsa unei chei API, serviciul poate folosi fallback local.

Implementarea comunicării în timp real

Pentru comunicarea realtime am implementat un manager WebSocket în backend. Utilizatorul se conectează cu token JWT, iar backend-ul îl asociază cu proiectele din care face parte. Frontend-ul are un bridge global care ascultă evenimentele și notifică paginile active.

Implementarea integrării GitHub

Integrarea GitHub este realizată printr-un endpoint de webhook. Cererea este verificată cu semnătură și secret configurat. După validare, backend-ul interpretează evenimentele de commit și pull request și actualizează task-urile corespunzătoare.

Implementarea raportării

Backend-ul calculează indicatorii pe baza task-urilor, sprinturilor și audit log-urilor. Frontend-ul afișează datele prin grafice, iar exportul PDF este generat server-side.

4.3 Validarea funcționalităților implementate

Pentru a verifica funcționalitatea aplicației, am folosit testare manuală pe fluxurile principale și verificări tehnice de build și pornire.

Verificări tehnice

După implementarea modulelor importante, am rulat verificări pentru backend și frontend: compilarea codului Python, rularea migrațiilor Alembic, lint pentru frontend, build pentru frontend, pornirea aplicației prin scriptul local și verificarea endpoint-ului de health.

Comenzile folosite au fost:

- `python -m compileall backend;`
- `alembic upgrade head;`
- `npm run lint;`
- `npm run build;`
- `./start.sh;`
- `curlhttp://127.0.0.1:8000/health.`

Scenarii de testare manuală

Am verificat manual fluxurile principale: înregistrare, verificare e-mail, autentificare, creare proiect, Project Wizard, creare task-uri, mutare pe board, creare și închidere sprint, schimbare metodologie, invitații și roluri, permisiuni, calendar, evenimente recurente, funcții AI, release notes, documentație, notificări, realtime, integrare GitHub și export PDF.

Validarea tranzițiilor între metodologii

Am testat trecerea de la Scrum la Kanban cu sprint activ, caz în care tranziția trebuie blocată, trecerea de la Scrum la Kanban după închiderea sprintului, reapariția Backlog-ului la revenirea spre Scrum sau Scrumban și ascunderea câmpurilor specifice sprinturilor în Kanban.

Rezultatele obținute

În urma implementării, aplicația oferă un flux complet de lucru pentru proiecte software. Utilizatorul poate crea un proiect, poate primi o metodologie recomandată, poate gestiona task-uri și sprinturi, poate colabora cu membrii echipei, poate primi notificări, poate urmări rapoarte și poate conecta activitatea din GitHub cu task-urile aplicației.

Prin aceste rezultate, contribuția proprie demonstrează proiectarea și implementarea unei aplicații practice, integrate și extensibile pentru managementul proiectelor software.

Bibliografie

- [1] Mariyam Mahboob, Mohammed Rayyan Uddin Ahmed, Zoiba Zia, Mariam Shakeel Ali, and Ayman Khaleel Ahmed. Future of artificial intelligence in agile software development. <https://arxiv.org/abs/2408.00703>, 2024.
- [2] Howard Lei, Farnaz Ganjeizadeh, Pradeep Kumar Jayachandran, and Pinar Ozcan. A statistical analysis of the effects of scrum and kanban on software development projects. *Robotics and Computer-Integrated Manufacturing*, 43:59–67, 2017.
- [3] Mohammad Amin Zadenoori, Jacek Dąbrowski, Waad Alhoshan, Liping Zhao, and Alessio Ferrari. Large language models (llms) for requirements engineering (re): A systematic literature review. <https://arxiv.org/abs/2509.11446>, 2025.
- [4] Yasir Mahmood, Nazri Kama, Azri Azmi, Ahmad Salman Khan, and Mazlan Ali. Software effort estimation accuracy prediction of machine learning techniques: A systematic performance evaluation. <https://arxiv.org/abs/2101.10658>, 2021.